LAPORAN TUGAS FRAKTAL MANDELBROT PEMROGRAMAN PARALEL



Dosen Pengampu:

Amir Murtako, S.Kom., M.Kom

Nama:

Nayunda Krisna / 4524210077

PROGRAM STUDI TEKNIK INFORMATIKA

UNIVERSITAS PANCASILA

2026

Source Code:

```
#include <chrono>
#include <iostream>
#include <omp.h>
#include <complex>
#include <cstdio>

typedef std::complex<double> complex;

int MandelbrotCalculate(complex c, int maxiter)
{
    complex z = c;
    int n = 0;
    for(; n < maxiter; ++n)
    {
        if(std::abs(z) >= 2.0) break;
        z = z * z + c;
    }
    return n;
}

void MandelBrotSerial()
{
    const int width = 78, height = 44, num_pixels = width *
height;

    const complex center(-.7, 0), span(2.7, -(4/3.0) * 2.7 *
height / width);
    const complex begin = center - span / 2.0;
    const int maxiter = 100000;

    for(int pix = 0; pix < num_pixels; ++pix)
    {
        const int x = pix % width, y = pix / width;

        complex c = begin + complex(x * span.real() / (width +
1.0),
                                    y * span.imag() / (height +
1.0));

        int n = MandelbrotCalculate(c, maxiter);
```

```
        if(n == maxiter) n = 0;

        {
            char c = ' ';
            if(n > 0)
            {
                static const char charset[] = "NAYUNDA";
                c = charset[n % (sizeof(charset) - 1)];
            }
            std::putchar(c);
            if(x + 1 == width) std::puts("|");
        }
    }
}

void MandelBrotParallel()
{
    const int width = 78, height = 44, num_pixels = width *
height;

    const complex center(-.7, 0), span(2.7, -(4/3.0) * 2.7 *
height / width);
    const complex begin = center - span / 2.0;
    const int maxiter = 100000;

    #pragma omp parallel for ordered schedule(dynamic)
    for(int pix = 0; pix < num_pixels; ++pix)
    {
        const int x = pix % width, y = pix / width;

        complex c = begin + complex(x * span.real() / (width +
1.0),
                                    y * span.imag() / (height +
1.0));

        int n = MandelbrotCalculate(c, maxiter);
        if(n == maxiter) n = 0;

        #pragma omp ordered
        {
            char c = ' ';
            if(n > 0)
```

```
            {
                static const char charset[] = "NAYUNDA";
                c = charset[n % (sizeof(charset) - 1)];
            }
            std::putchar(c);
            if(x + 1 == width) std::puts("|");
        }
    }
}

int main()
{
    std::cout << "---------------------------------" <<
std::endl;
    std::cout << "NAMA : NAYUNDA KRISNA ABI PASYA" << std::endl;
    std::cout << "NPM  : 4524210077" << std::endl;
    std::cout << "---------------------------------" <<
std::endl;

    int m = omp_get_max_threads();
    std::cout << "Number of threads\t" << m << ".\n\n";

    auto start_time = std::chrono::high_resolution_clock::now();
    MandelBrotSerial();
    auto end_time = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> serial_duration = end_time -
start_time;

    start_time = std::chrono::high_resolution_clock::now();
    MandelBrotParallel();
    end_time = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> parallel_duration = end_time -
start_time;

    std::cout << "---------------------------------" <<
std::endl;
    std::cout << "Serial duration: " << serial_duration.count()
<< " seconds" << std::endl;
    std::cout << "Parallel duration: " <<
parallel_duration.count() << " seconds" << std::endl;
    std::cout << "Speedup: " << serial_duration.count() /
parallel_duration.count() << std::endl;
```

```
    std::cout << "-----------------------------------" <<
std::endl;

    return 0;
}
```

Hasil Compile:

